

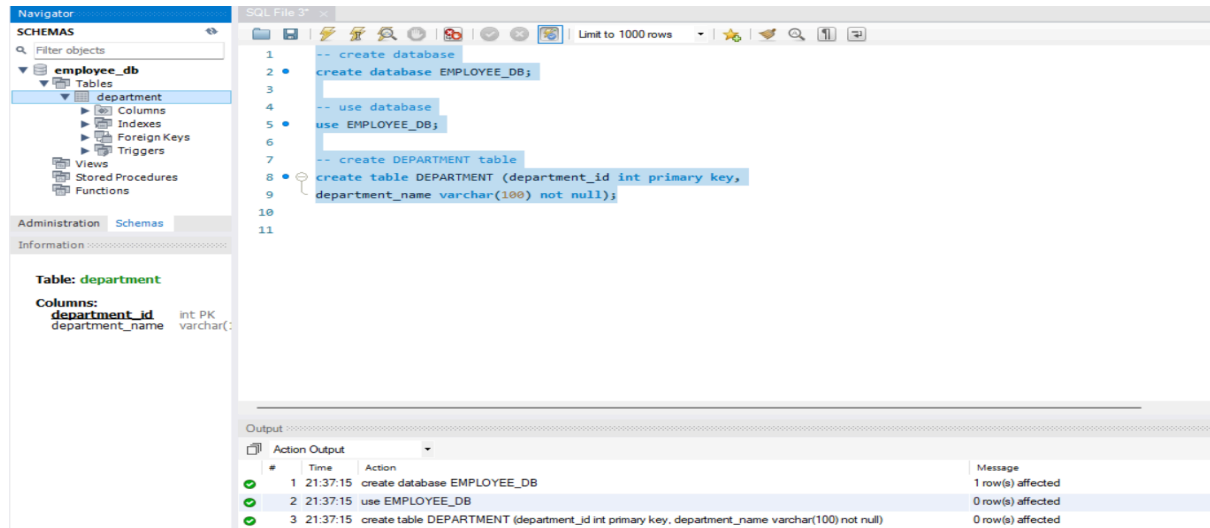
MODULE -3

MYSQL ASSIGNMENT 1

DDL COMMANDS, CONSTRAINTS

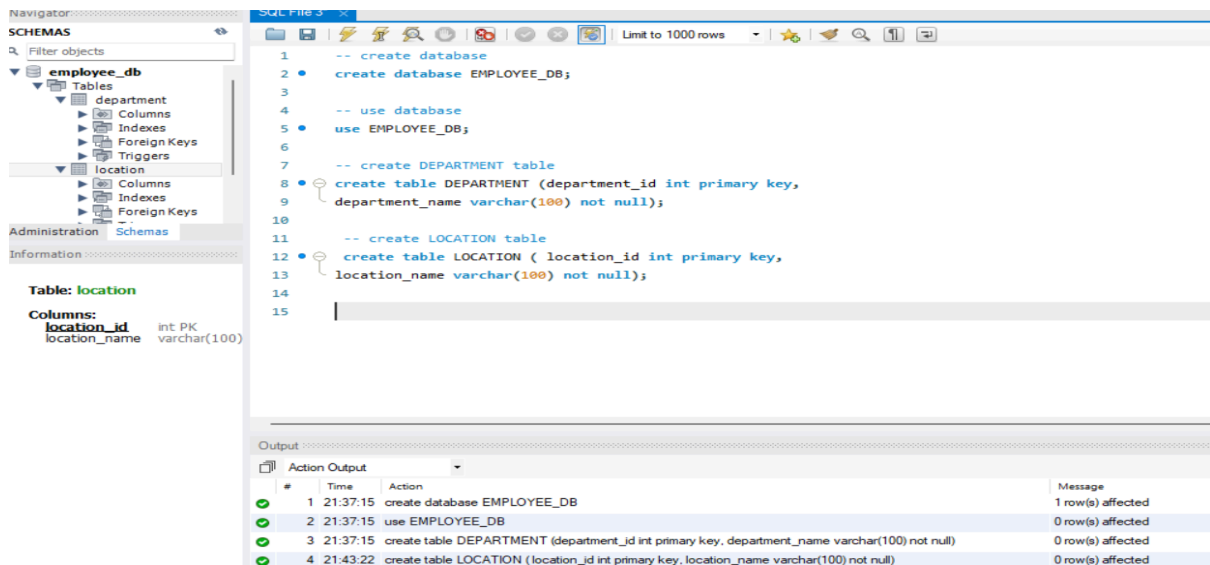
1.Table: Departments Attributes:

department_id – Primary Key , department_name



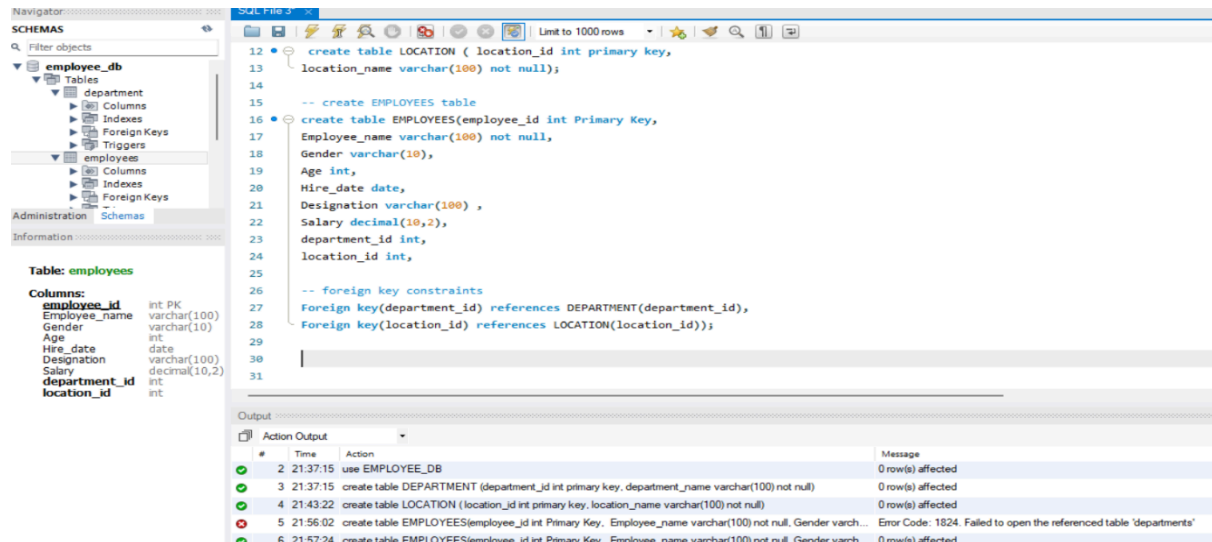
2.Table: Location Attributes:

location_id – Primary Key & location_name



3.Table: Employees Attributes:

- employee_id – Primary Key
- Employee_name
- Gender
- Age
- Hire_date
- Designation
- Salary
- department_id – Foreign Key referencing department(department_id)
- location_id—Foreign Key referencing location(location_id)

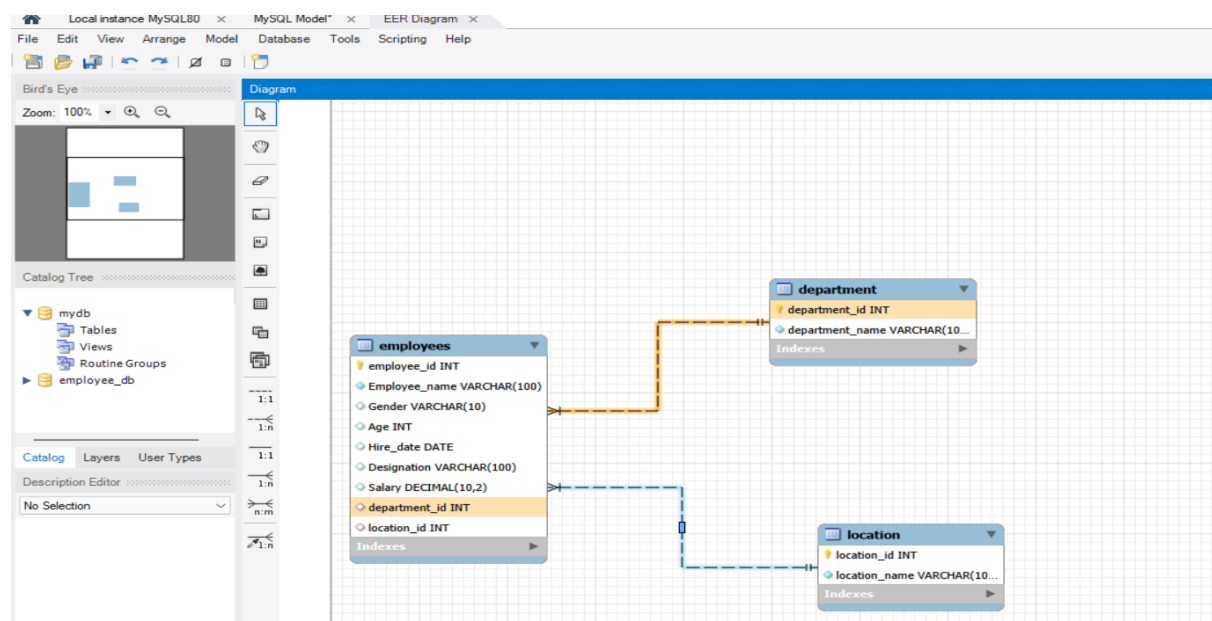


Tasks:

DDL Commands

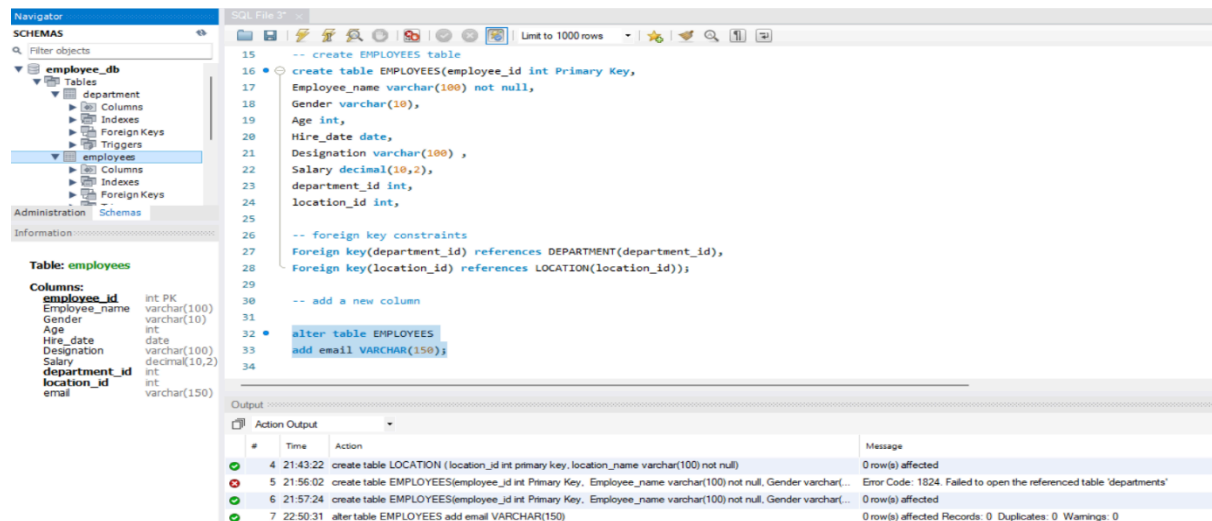
1. Database and Table Creation (CREATE):

- Create a Database for this project
- Create all three tables in MySQL with appropriate data types and relationships.

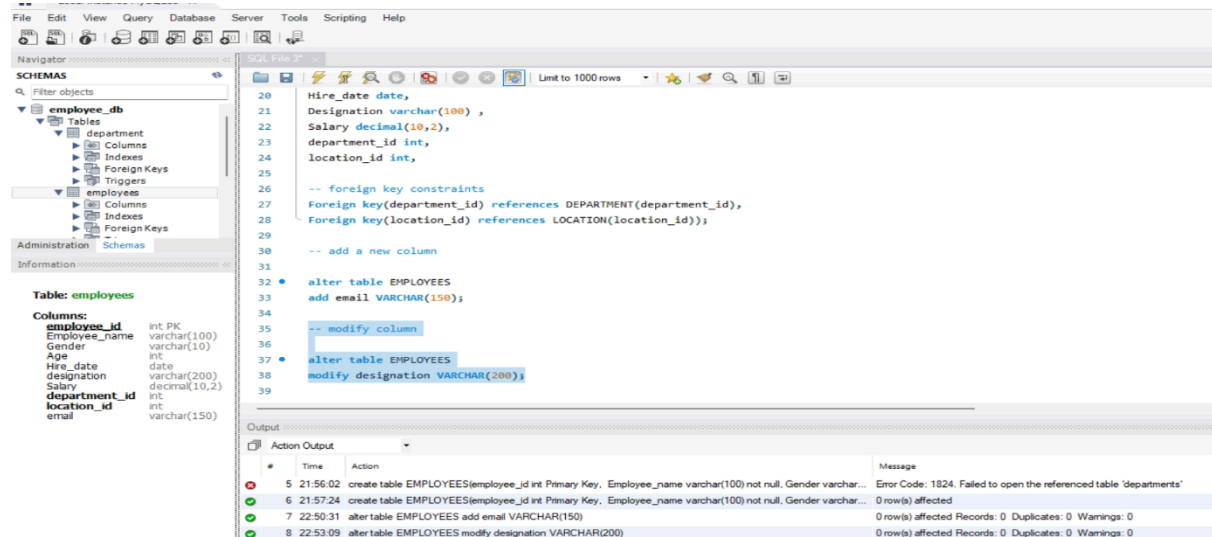


2. Table Alteration (ALTER)

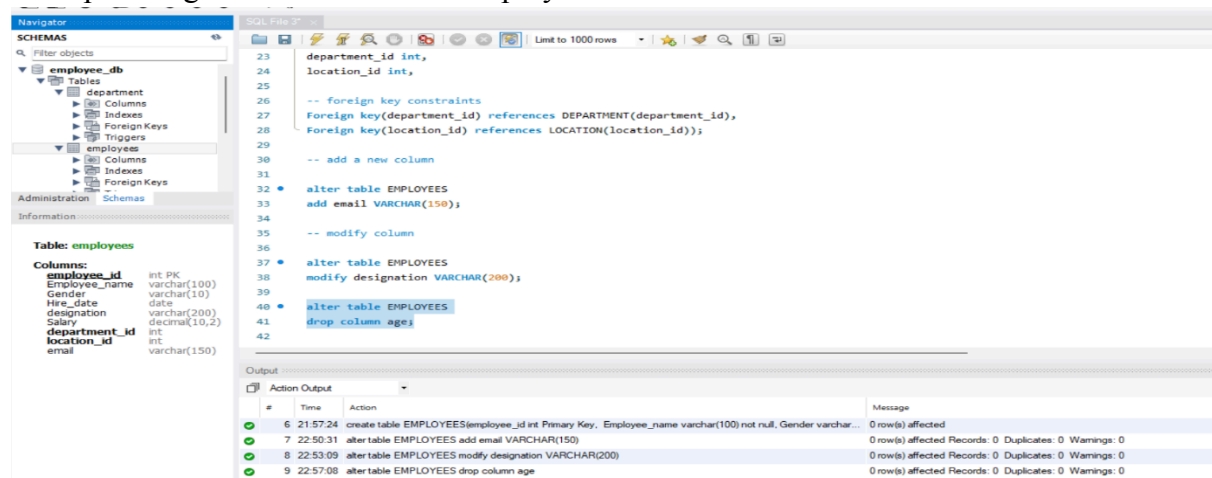
- Add a new column named "email" to the Employees table to store employee email addresses.



- Modify the data type of the "designation" column in the Employees table to support a wider range of values.



- Drop the "age" column from the Employees table.



- Rename the “hire_date” column to “date_of_joining”.

The screenshot shows the SQL Developer interface. On the left, the 'Schemas' pane shows the 'employee_db' schema with tables 'department' and 'employees'. The 'employees' table is selected, and its columns are listed: employee_id (int PK), employee_name (varchar(100)), gender (varchar(10)), date_of_joining (date), designation (varchar(200)), salary (decimal(10,2)), department_id (int), location_id (int), and email (varchar(150)).

The main SQL editor shows the following SQL script:

```

26 -- foreign key constraints
27 Foreign key(department_id) references DEPARTMENT(department_id);
28 Foreign key(location_id) references LOCATION(location_id));
29
30 -- add a new column
31
32 • alter table EMPLOYEES
33   add email VARCHAR(150);
34
35 -- modify column
36
37 • alter table EMPLOYEES
38   modify designation VARCHAR(200);
39
40 • alter table EMPLOYEES
41   drop column age;
42
43 • alter table EMPLOYEES
44   rename column Hire_date to date_of_joining;
45

```

The 'Output' pane shows the execution results:

#	Time	Action	Message
7	22:50:31	alter table EMPLOYEES add email VARCHAR(150)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
8	22:53:09	alter table EMPLOYEES modify designation VARCHAR(200)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
9	22:57:08	alter table EMPLOYEES drop column age	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
10	23:01:01	alter table EMPLOYEES rename column Hire_date to date_of_joining	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0

3. Table Renaming (RENAME):

- Rename the "Departments" table to "Departments_Info".

The screenshot shows the SQL Developer interface. On the left, the 'Schemas' pane shows the 'employee_db' schema with tables 'department' and 'employees'. The 'department' table is selected, and its columns are listed: department_id (int PK) and department_name (varchar(100)).

The main SQL editor shows the following SQL script:

```

35 -- modify column
36
37 • alter table EMPLOYEES
38   modify designation VARCHAR(200);
39
40 • alter table EMPLOYEES
41   drop column age;
42
43 • alter table EMPLOYEES
44   rename column Hire_date to date_of_joining;
45
46
47
48
49 -- rename department
50
51 Rename table DEPARTMENT to DEPARTMENTS_INFO;
52
53
54

```

The 'Output' pane shows the execution results:

#	Time	Action	Message
8	22:53:09	alter table EMPLOYEES modify designation VARCHAR(200)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
9	22:57:08	alter table EMPLOYEES drop column age	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
10	23:01:01	alter table EMPLOYEES rename column Hire_date to date_of_joining	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
11	23:07:32	Rename table DEPARTMENT to DEPARTMENTS_INFO	0 row(s) affected

- Rename the "Location" table to "Locations".

The screenshot shows the SQL Developer interface. On the left, the 'Schemas' pane shows the 'employee_db' schema with tables 'department' and 'employees'. The 'location' table is selected, and its columns are listed: location_id (int PK) and location_name (varchar(100)).

The main SQL editor shows the following SQL script:

```

35 -- modify column
36
37 • alter table EMPLOYEES
38   modify designation VARCHAR(200);
39
40 • alter table EMPLOYEES
41   drop column age;
42
43 • alter table EMPLOYEES
44   rename column Hire_date to date_of_joining;
45
46
47
48
49 -- rename department
50
51 Rename table DEPARTMENT to DEPARTMENTS_INFO;
52
53 • Rename table LOCATION to LOCATIONS;
54

```

The 'Output' pane shows the execution results:

#	Time	Action	Message
9	22:57:08	alter table EMPLOYEES drop column age	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
10	23:01:01	alter table EMPLOYEES rename column Hire_date to date_of_joining	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
11	23:07:32	Rename table DEPARTMENT to DEPARTMENTS_INFO	0 row(s) affected
12	23:09:20	Rename table LOCATION to LOCATIONS	0 row(s) affected

4. Table Truncation (TRUNCATE):

- Truncate the Employees table.

```

57  -- table truncate
58
59  • truncate table EMPLOYEES;
60
61
62
63

```

#	Time	Action	Message
✓ 10	23:01:01	alter table EMPLOYEES rename column Hire_date to date_of_joining	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
✓ 11	23:07:32	Rename table DEPARTMENT to DEPARTMENTS_INFO	0 row(s) affected
✓ 12	23:09:20	Rename table LOCATION to LOCATIONS	0 row(s) affected
✓ 13	23:11:57	truncate table EMPLOYEES	0 row(s) affected

5. Database & Table Dropping (DROP):

- Drop the Employees table and then the “employee” database.

#	Time	Action	Message
✓ 11	23:07:32	Rename table DEPARTMENT to DEPARTMENTS_INFO	0 row(s) affected
✓ 12	23:09:20	Rename table LOCATION to LOCATIONS	0 row(s) affected
✓ 13	23:11:57	truncate table EMPLOYEES	0 row(s) affected
✓ 14	23:15:15	drop table EMPLOYEES	0 row(s) affected

#	Time	Action	Message
✓ 12	23:09:20	Rename table LOCATION to LOCATIONS	0 row(s) affected
✓ 13	23:11:57	truncate table EMPLOYEES	0 row(s) affected
✓ 14	23:15:15	drop table EMPLOYEES	0 row(s) affected
✓ 15	23:23:44	drop database EMPLOYEE_DB	2 row(s) affected

Tasks:

Constraints –

1. Database Recreation:

- Drop the 'employee' database if it exists
- Recreate it using the provided schema, ensuring that all tables are created with the appropriate constraints as instructed.
- Establish links between the "department_id" and "location_id" fields in the "employees" table and their respective tables.

The screenshot displays the SQL Server Enterprise Manager interface. On the left, the 'Schemas' pane shows the 'employee_db' database with tables 'department', 'employees', and 'location'. The 'employees' table is selected, and its structure is shown in the 'Table: employees' pane. The columns are: employee_id (int PK), Employee_name (varchar(100)), Gender (char(1)), Age (int), Hire_date (date), Designation (varchar(200)), Salary (decimal(10,2)), email (varchar(150)), department_id (int), and location_id (int). The main pane shows the SQL script for creating the database and tables.

```
1 create database EMPLOYEE_DB;
2
3
4 -- use database
5 use EMPLOYEE_DB;
6
7 -- create DEPARTMENT table
8 create table DEPARTMENT (department_id int primary key,
9 department_name varchar(100) not null UNIQUE);
10
11 -- create LOCATION table
12 create table LOCATION (
13 location_id int AUTO_INCREMENT primary key,
14 location_name varchar(100) not null UNIQUE);
15
16 -- create EMPLOYEES table
17 create table EMPLOYEES (employee_id int Primary Key,
18 Employee_name varchar(100) not null,
19 Gender CHAR(1) CHECK (Gender IN ('M', 'F')),
20 Age int CHECK (Age >= 18),
21 Hire_date date DEFAULT (CURRENT_DATE),
22 Designation varchar(200),
23 Salary decimal(10,2),
24 email VARCHAR(150),
25 department_id int,
26 location_id int,
27
28 -- foreign key constraints
29 Foreign key(department_id) references DEPARTMENT(department_id),
30 Foreign key(location_id) references LOCATION(location_id);
31
```

2. Departments Table:

- Ensure that the "department_id" uniquely identifies each department.
- Set up constraints on the "department_name" to avoid duplicate and null entries.

The screenshot displays the SQL Server Enterprise Manager interface. On the left, the 'Schemas' pane shows the 'employee_db' database with tables 'department', 'employees', and 'location'. The 'department' table is selected, and its structure is shown in the 'Table: department' pane. The columns are: department_id (int PK) and department_name (varchar(100)). The main pane shows the SQL script for creating the database and the 'department' table.

```
1 -- create database
2 create database EMPLOYEE_DB;
3
4 -- use database
5 use EMPLOYEE_DB;
6
7 -- create DEPARTMENT table
8 create table DEPARTMENT (department_id int primary key,
9 department_name varchar(100) not null UNIQUE);
10
```

The 'Output' pane at the bottom shows the execution results:

#	Time	Action	Message
15	23:23:44	drop database EMPLOYEE_DB	2 row(s) affected
16	23:27:29	create database EMPLOYEE_DB	1 row(s) affected
17	23:27:29	use EMPLOYEE_DB	0 row(s) affected
18	23:27:29	create table DEPARTMENT (department_id int primary key, department_name varchar(100) not null UNIQUE)	0 row(s) affected

3. Locations Table:

- Establish a mechanism to automatically generate unique identifiers for each location, ensuring that they are incremented sequentially.

- Implement constraints to prevent the insertion of null and duplicate locations.

SQL File 1

```

6
7  -- create DEPARTMENT table
8  create table DEPARTMENT (department_id int primary key,
9  department_name varchar(100) not null UNIQUE);
10
11  -- create LOCATION table
12  create table LOCATION (
13  location_id int AUTO_INCREMENT primary key,
14  location_name varchar(100) not null UNIQUE);
15

```

Table: location

Columns:

Column Name	Column Type	Column Constraints
location_id	int	AI PK
location_name	varchar(100)	

Action Output

#	Time	Action	Message
16	23:27:29	create database EMPLOYEE_DB	1 row(s) affected
17	23:27:29	use EMPLOYEE_DB	0 row(s) affected
18	23:27:29	create table DEPARTMENT (department_id int primary key, department_name varchar(100) not null UNIQUE)	0 row(s) affected
19	23:36:47	create table LOCATION (location_id int AUTO_INCREMENT primary key, location_name varchar(100) not null ...)	0 row(s) affected

4. Employees Table:

- Guarantee that each employee has a distinct identifier.
- Create a restriction to ensure that the employee's name is always provided.
- Limit the acceptable values for the "gender" field to only 'M' or 'F'.
- Enforce a condition to ensure that the employee's age is 18 or above.
- Automatically assign the current date to the "hire_date" field if not specified.

SQL File 1

```

12  create table LOCATION (
13  location_id int AUTO_INCREMENT primary key,
14  location_name varchar(100) not null UNIQUE);
15
16  -- create EMPLOYEES table
17  create table EMPLOYEES(employee_id int Primary Key,
18  Employee_name varchar(100) not null,
19  Gender CHAR(1) CHECK (Gender IN ('M', 'F')),
20  Age int CHECK (age >= 18),
21  Hire_date date DEFAULT (CURRENT_DATE),
22  Designation varchar(200) ,
23  Salary decimal(10,2),
24  email VARCHAR(150),
25  department_id int,
26  location_id int,
27
28  -- foreign key constraints
29  Foreign key(department_id) references DEPARTMENT(department_id),
30  Foreign key(location_id) references LOCATION(location_id));
31
32  -- add a new column

```

Table: employees

Columns:

Column Name	Column Type	Column Constraints
employee_id	int	PK
Employee_name	varchar(100)	
Gender	char(1)	
Age	int	
Hire_date	date	
Designation	varchar(200)	
Salary	decimal(10,2)	
email	varchar(150)	
department_id	int	
location_id	int	